

```
!pip install --upgrade torch torchvision sympy
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.12.0)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.27.0)
Requirement already satisfied: sympy in /usr/local/lib/python3.12/dist-packages (1.14.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: cuda-toolkit==13.0.2 in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cudart,cu
Requirement already satisfied: nvidia-cublas<=13.1.1.3,>=13.1.0.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1
Requirement already satisfied: cuda-bindings<14,>=13.0.3 in /usr/local/lib/python3.12/dist-packages (from torch) (13.3.1)
Requirement already satisfied: nvidia-cudnn-cu13==9.20.0.48 in /usr/local/lib/python3.12/dist-packages (from torch) (9.20.0.
Requirement already satisfied: nvidia-cusparselt-cu13==0.8.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.8.1)
Requirement already satisfied: nvidia-nccl-cu13==2.29.7 in /usr/local/lib/python3.12/dist-packages (from torch) (2.29.7)
Requirement already satisfied: nvidia-nvshmem-cu13==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: triton==3.7.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.7.0)
Requirement already satisfied: nvidia-cuda-runtime==13.0.96.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[
Requirement already satisfied: nvidia-cufft==12.0.0.61.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cudar
Requirement already satisfied: nvidia-cuda-cupti==13.0.85.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cu
Requirement already satisfied: nvidia-curand==10.4.0.35.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cuda
Requirement already satisfied: nvidia-cusolver==12.0.4.66.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cu
Requirement already satisfied: nvidia-cusparselt==12.6.3.3.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cuc
Requirement already satisfied: nvidia-nvjitlink==13.0.88.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cuc
Requirement already satisfied: nvidia-cuda-nvrtc==13.0.88.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cu
Requirement already satisfied: nvidia-nvtx==13.0.85.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cudart,c
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy) (1.3.0)
Requirement already satisfied: cuda-pathfinder>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings<14,>=13
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
```

✓ Entrega Parcial 3 – Desarrollo Avanzado del Sistema

Proyecto: PlantDoc – Clasificación Inteligente de Patologías Vegetales



Modelo Seleccionado: ResNet-50 con Transfer Learning

Para clasificar **39 categorías** de enfermedades vegetales, **ResNet-50** es la arquitectura óptima:

Arquitectura	¿Aplica?	Motivo
ResNet-50	✅ Seleccionada	Clasificación multiclase, <i>skip connections</i> eliminan el vanishing gradient, 25M parámetros vs 138M de VGG, estado del arte en PlantVillage
VGG-16	⚠ Descartada	Válida para clasificación pero 138M parámetros penalizan el entrenamiento sin mejora real de precisión
U-Net	❌ No aplica	Diseñada para segmentación semántica píxel a píxel, no clasificación
YOLO	❌ No aplica	Detección de objetos con bounding boxes, problema distinto
ViT	⚠ Descartada	Requiere millones de imágenes para superar CNNs; sobreajustaría en este dataset
DETR	❌ No aplica	Detección de objetos con Transformers

Estrategia aplicada: Transfer Learning con pesos ImageNet. Se congelan las capas `conv1` → `layer3` (características genéricas) y se hace fine-tuning sobre `layer4` + nueva cabeza `fc` adaptada a 39 clases.

```
# -*- coding: utf-8 -*-
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from PIL import Image
import pandas as pd
import random
import warnings
warnings.filterwarnings('ignore')

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms, models

from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
from tqdm.auto import tqdm
```

```
# Reproducibilidad
random.seed(42)
np.random.seed(42)
torch.manual_seed(42)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Dispositivo disponible: {device}')
```

Dispositivo disponible: cuda

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
# Descarga del dataset desde Google Drive
!gdown 1Bru37ZY-BXGP89w578vVPCsnQyrVXFVs

import zipfile

zip_path = '/content/Plant_leaf_diseases_dataset_without_augmentation.zip'
extract_path = '/content/dataset_plantas'

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print('Dataset descomprimido correctamente')
```

Downloading...
 From (original): <https://drive.google.com/uc?id=1Bru37ZY-BXGP89w578vVPCsnQyrVXFVs>
 From (redirected): <https://drive.google.com/uc?id=1Bru37ZY-BXGP89w578vVPCsnQyrVXFVs&confirm=t&uid=2dd48456-7262-415d-9ab8-c>
 To: /content/Plant_leaf_diseases_dataset_without_augmentation.zip
 100% 868M/868M [00:16<00:00, 51.1MB/s]
 Dataset descomprimido correctamente

```
dataset_path = '/content/dataset_plantas/Plant_leave_diseases_dataset_without_augmentation'
```

```
clases = sorted([
    c for c in os.listdir(dataset_path)
    if os.path.isdir(os.path.join(dataset_path, c))
])

print(f'Total de clases: {len(clases)}')
print('Primeras 10 clases:', clases[:10])
```

Total de clases: 39
 Primeras 10 clases: ['Apple__Apple_scab', 'Apple__Black_rot', 'Apple__Cedar_apple_rust', 'Apple__healthy', 'Background_w']

✓ 1. Exploración del Dataset

```
conteo_clases = {}
for clase in clases:
    carpeta = os.path.join(dataset_path, clase)
    imgs = [f for f in os.listdir(carpeta)
             if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    conteo_clases[clase] = len(imgs)

df_clases = pd.DataFrame(
    list(conteo_clases.items()),
    columns=['Clase', 'Cantidad']
).sort_values('Cantidad', ascending=False).reset_index(drop=True)

print(f'Total de imagenes en el dataset: {df_clases["Cantidad"].sum():,}')
display(df_clases)
```

Total de imagenes en el dataset: 55,448

	Clase	Cantidad
0	Orange___Haunglongbing_(Citrus_greening)	5507
1	Tomato___Tomato_Yellow_Leaf_Curl_Virus	5357
2	Soybean___healthy	5090
3	Peach___Bacterial_spot	2297
4	Tomato___Bacterial_spot	2127
5	Tomato___Late_blight	1909
6	Squash___Powdery_mildew	1835
7	Tomato___Septoria_leaf_spot	1771
8	Tomato___Spider_mites Two-spotted_spider_mite	1676
9	Apple___healthy	1645
10	Tomato___healthy	1591
11	Blueberry___healthy	1502
12	Pepper_bell___healthy	1478
13	Tomato___Target_Spot	1404
14	Grape___Esca_(Black_Measles)	1383
15	Corn___Common_rust	1192
16	Grape___Black_rot	1180
17	Corn___healthy	1162
18	Background_without_leaves	1143
19	Strawberry___Leaf_scorch	1109
20	Grape___Leaf_blight_(Isariopsis_Leaf_Spot)	1076
21	Cherry___Powdery_mildew	1052
22	Tomato___Early_blight	1000
23	Potato___Early_blight	1000
24	Potato___Late_blight	1000
25	Pepper_bell___Bacterial_spot	997
26	Corn___Northern_Leaf_Blight	985
27	Tomato___Leaf_Mold	952
28	Cherry___healthy	854
29	Apple___Apple_scab	630
30	Apple___Black_rot	621
31	Corn___Cercospora_leaf_spot Gray_leaf_spot	513
32	Strawberry___healthy	456
33	Grape___healthy	423
34	Tomato___Tomato_mosaic_virus	373
35	Raspberry___healthy	371
36	Peach___healthy	360
37	Apple___Cedar_apple_rust	275
38	Potato___healthy	152

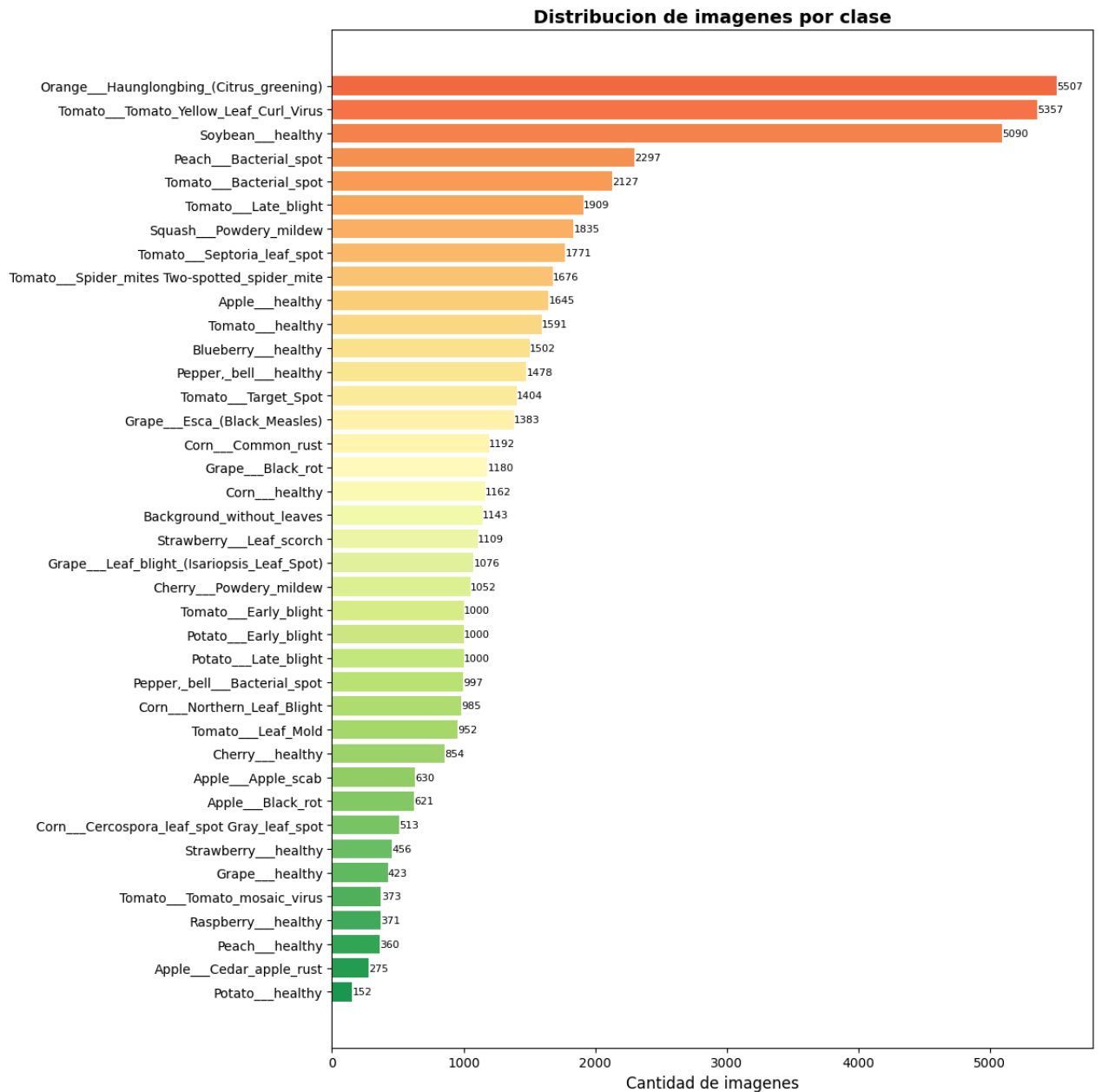
```
fig, ax = plt.subplots(figsize=(12, 12))

colors = plt.cm.RdYlGn(np.linspace(0.2, 0.9, len(df_clases)))
bars = ax.barh(df_clases['Clase'], df_clases['Cantidad'], color=colors)

ax.set_xlabel('Cantidad de imagenes', fontsize=12)
ax.set_title('Distribucion de imagenes por clase', fontsize=14, fontweight='bold')
ax.invert_yaxis()

for bar, val in zip(bars, df_clases['Cantidad']):
    ax.text(bar.get_width() + 2, bar.get_y() + bar.get_height() / 2,
            str(val), va='center', fontsize=8)
```

```
plt.tight_layout()
plt.show()
```



2. Preprocesamiento de Imagenes (*Herencia Entrega 2*)

Se conserva el pipeline de procesamiento clasico. Estas tecnicas sirven como herramienta de analisis exploratorio visual antes de delegar el preprocesamiento a las transformaciones del modelo.

```
def procesar_imagen(ruta_imagen):
    """Pipeline completo de procesamiento clasico de imagen."""
    img_bgr = cv2.imread(ruta_imagen)
    if img_bgr is None:
        return None

    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
    img_blur = cv2.GaussianBlur(img_gray, (5, 5), 0)
    bordes = cv2.Canny(img_blur, 50, 150)

    kernel = np.ones((5, 5), np.uint8)
    dilatada = cv2.dilate(bordes, kernel, iterations=1)
    _, segmentada = cv2.threshold(img_gray, 120, 255, cv2.THRESH_BINARY_INV)

    return img_rgb, img_gray, img_blur, bordes, dilatada, segmentada
```

```

etiquetas = ['Original', 'Grises', 'Blur', 'Canny', 'Dilatacion', 'Segmentacion']
muestras = []

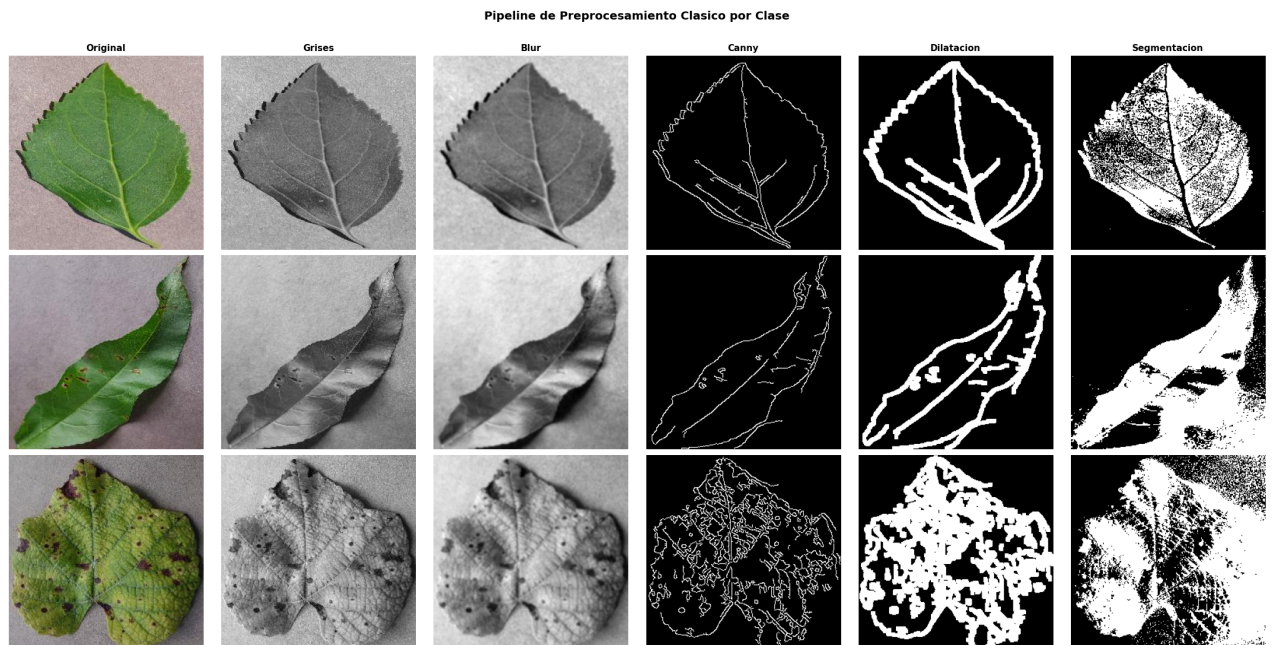
for _ in range(3):
    clase = random.choice(clases)
    carpeta = os.path.join(dataset_path, clase)
    archivos = [f for f in os.listdir(carpeta)
                 if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    result = procesar_imagen(os.path.join(carpeta, random.choice(archivos)))
    if result:
        muestras.append((clase, result))

fig, axes = plt.subplots(3, 6, figsize=(22, 11))

for i, (clase, etapas) in enumerate(muestras):
    for j, (etapa, etiqueta) in enumerate(zip(etapas, etiquetas)):
        ax = axes[i, j]
        ax.imshow(etapa, cmap=(None if j == 0 else 'gray'))
        if i == 0:
            ax.set_title(etiqueta, fontsize=11, fontweight='bold')
        if j == 0:
            ax.set_ylabel(clase[:28], fontsize=7.5, rotation=0, labelpad=90, va='center')
        ax.axis('off')

plt.suptitle('Pipeline de Preprocesamiento Clasico por Clase', fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()

```



3. Preparacion del Dataset para Deep Learning

Se definen dos conjuntos de transformaciones:

- **Entrenamiento:** Resize 256x256 → RandomCrop 224x224 + augmentations (flip, rotacion, color jitter) + normalizacion ImageNet
- **Validacion / Test:** Resize 224x224 + normalizacion ImageNet (sin augmentations para evaluacion objetiva)

Division del dataset: 70% train · 15% validacion · 15% test

```

transform_train = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(p=0.2),
    transforms.RandomRotation(20),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2, hue=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

transform_eval = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

```

```

transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# Tres instancias del dataset (mismas imagenes, distintos transforms)
full_train = datasets.ImageFolder(dataset_path, transform=transform_train)
full_eval = datasets.ImageFolder(dataset_path, transform=transform_eval)

class_names = full_train.classes
total = len(full_train)

# Indices aleatorios con semilla fija
indices = list(range(total))
random.shuffle(indices)

train_end = int(0.70 * total)
val_end = int(0.85 * total)

train_indices = indices[:train_end]
val_indices = indices[train_end:val_end]
test_indices = indices[val_end:]

train_ds = Subset(full_train, train_indices)
val_ds = Subset(full_eval, val_indices)
test_ds = Subset(full_eval, test_indices)

print(f'Clases encontradas: {len(class_names)}')
print(f'Total: {total:,} imagenes')
print(f'Train: {len(train_ds):,} ({len(train_ds)/total:.0%})')
print(f'Validacion: {len(val_ds):,} ({len(val_ds)/total:.0%})')
print(f'Test: {len(test_ds):,} ({len(test_ds)/total:.0%})')

```

```

Clases encontradas: 39
Total: 55,448 imagenes
Train: 38,813 (70%)
Validacion: 8,317 (15%)
Test: 8,318 (15%)

```

```

BATCH_SIZE = 32

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,
                           num_workers=2, pin_memory=True)
val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False,
                        num_workers=2, pin_memory=True)
test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False,
                          num_workers=2, pin_memory=True)

print(f'Batches por epoca (train): {len(train_loader)}')
print(f'Batches por epoca (val): {len(val_loader)}')
print(f'Batches por epoca (test): {len(test_loader)}')

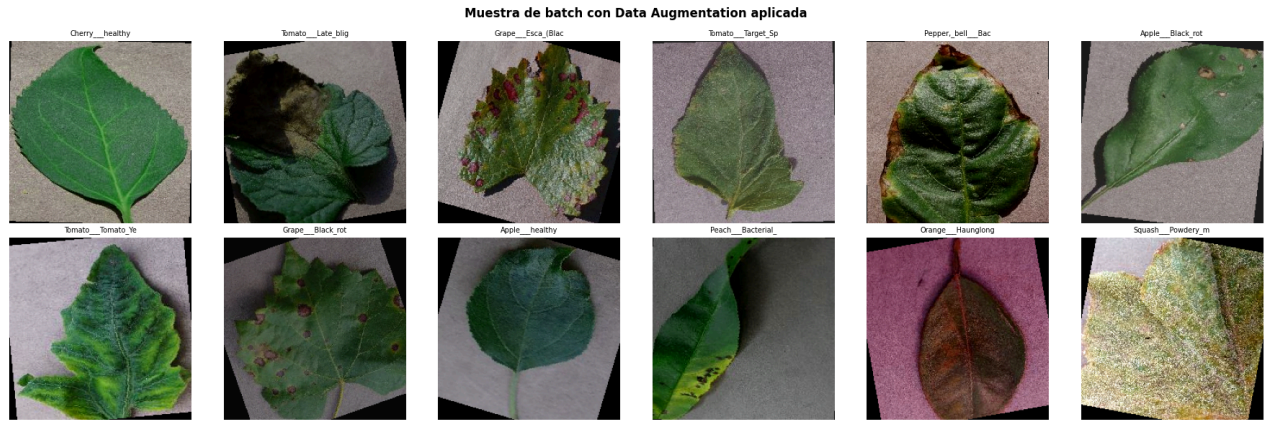
# Muestra visual de un batch con augmentation
inv_norm = transforms.Normalize(
    mean=[-0.485/0.229, -0.456/0.224, -0.406/0.225],
    std=[1/0.229, 1/0.224, 1/0.225]
)
images_s, labels_s = next(iter(train_loader))

fig, axes = plt.subplots(2, 6, figsize=(18, 6))
for i, ax in enumerate(axes.flatten()):
    if i >= len(images_s):
        ax.axis('off')
        continue
    img = inv_norm(images_s[i]).permute(1, 2, 0).numpy()
    img = np.clip(img, 0, 1)
    ax.imshow(img)
    ax.set_title(class_names[labels_s[i]][:18], fontsize=7)
    ax.axis('off')

plt.suptitle('Muestra de batch con Data Augmentation aplicada', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()

```


Batches por epoca (train): 1213
 Batches por epoca (val): 260
 Batches por epoca (test): 260



4. Modelo ResNet-50 con Transfer Learning

Configuracion de capas:

Bloque	Estado	Descripcion
conv1, bn1, layer1, layer2, layer3	Congeladas	Extraccion de caracteristicas genericas de ImageNet
layer4	Fine-tuning	Caracteristicas de alto nivel adaptadas al dominio
fc (nueva)	Entrenamiento completo	Dropout(0.5) + Linear(2048 → 39 clases)

Esta estrategia preserva el conocimiento de bajo nivel (bordes, texturas, formas) aprendido en ImageNet y adapta las capas superiores al dominio especifico de hojas vegetales.

```
# Cargar ResNet-50 preentrenada en ImageNet
try:
    model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
except AttributeError:
    model = models.resnet50(pretrained=True)

# Paso 1: Congelar todas las capas
for param in model.parameters():
    param.requires_grad = False

# Paso 2: Descongelar layer4 para fine-tuning
for param in model.layer4.parameters():
    param.requires_grad = True

# Paso 3: Reemplazar la capa final (2048 -> 39 clases)
num_features = model.fc.in_features # 2048 en ResNet-50
model.fc = nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(num_features, len(class_names))
)

model = model.to(device)

# Resumen de parametros
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
frozen_params = total_params - trainable_params

print(f'Parametros totales: {total_params:,}')
print(f'Parametros entrenables: {trainable_params:,} ({trainable_params/total_params:.1%})')
print(f'Parametros congelados: {frozen_params:,} ({frozen_params/total_params:.1%})')
print(f'\nCabeza clasificadora:')
print(model.fc)
```

Downloading: "<https://download.pytorch.org/models/resnet50-11ad3fa6.pth>" to /root/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth [100%|██████████| 97.8M/97.8M [00:00<00:00, 195MB/s]]

Parametros totales: 23,587,943
 Parametros entrenables: 15,044,647 (63.8%)
 Parametros congelados: 8,543,296 (36.2%)

Cabeza clasificadora:
 Sequential(
 (0): Dropout(p=0.5, inplace=False)
 (1): Linear(in_features=2048, out_features=39, bias=True)
)

```

criterion = nn.CrossEntropyLoss()

# Tasas diferenciadas: menor en layer4 (ya entrenada), mayor en fc (nueva)
optimizer = optim.Adam([
    {'params': model.layer4.parameters(), 'lr': 1e-4, 'weight_decay': 1e-4},
    {'params': model.fc.parameters(), 'lr': 1e-3, 'weight_decay': 1e-4}
])

# Reduce LR si val_loss no mejora en 3 epocas consecutivas
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=3
)

print('Configuracion del optimizador:')
for i, group in enumerate(optimizer.param_groups):
    print(f' Grupo {i}: lr = {group["lr"]}')

Configuracion del optimizador:
Grupo 0: lr = 0.0001
Grupo 1: lr = 0.001

```

5. Entrenamiento del Modelo

```

def train_epoch(model, loader, criterion, optimizer, device):
    """Una epoca de entrenamiento con backpropagation."""
    model.train()
    running_loss, correct, total = 0.0, 0, 0

    for images, labels in tqdm(loader, desc='Train', leave=False):
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

    return running_loss / len(loader), 100.0 * correct / total

def eval_epoch(model, loader, criterion, device):
    """Una epoca de evaluacion sin gradientes."""
    model.eval()
    running_loss, correct, total = 0.0, 0, 0

    with torch.no_grad():
        for images, labels in tqdm(loader, desc='Eval', leave=False):
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)

            running_loss += loss.item()
            _, predicted = outputs.max(1)
            total += labels.size(0)
            correct += predicted.eq(labels).sum().item()

    return running_loss / len(loader), 100.0 * correct / total

```

```

import os

EPOCHS = 10

history = {
    'train_loss': [], 'val_loss': [],
    'train_acc': [], 'val_acc': []
}

best_val_acc = 0.0
# Carpeta para no llenar tu directorio principal de archivos sueltos
checkpoint_dir = '/content/checkpoints'
os.makedirs(checkpoint_dir, exist_ok=True)

best_model_path = os.path.join(checkpoint_dir, 'plantdoc_resnet50_best.pth')

```



```

print(f'Entrenando ResNet-50 en {device} durante {EPOCHS} epocas...')
print('-' * 62)
print(f'   Ep   | Train Loss | Train Acc | Val Loss | Val Acc')
print('-' * 62)

for epoch in range(1, EPOCHS + 1):
    train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer, device)
    val_loss, val_acc = eval_epoch(model, val_loader, criterion, device)

    scheduler.step(val_loss)

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['train_acc'].append(train_acc)
    history['val_acc'].append(val_acc)

    # 1. Estructura el diccionario con TODO lo necesario para replicar el estado actual
    checkpoint = {
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'scheduler_state_dict': scheduler.state_dict(),
        'history': history,
        'best_val_acc': best_val_acc
    }

    # 2. Guardar el checkpoint único de esta época
    epoch_path = os.path.join(checkpoint_dir, f'checkpoint_epoch_{epoch}.pth')
    torch.save(checkpoint, epoch_path)

    # 3. Guardar el "mejor modelo" de respaldo si supera el récord
    marker = ''
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        checkpoint['best_val_acc'] = best_val_acc # Actualizamos el valor en el diccionario
        torch.save(checkpoint, best_model_path)
        marker = ' <- mejor'

    print(f'   {epoch:2d}/{EPOCHS} | {train_loss:10.4f} | {train_acc:8.2f}% | '
          f'{val_loss:8.4f} | {val_acc:7.2f}%{marker}')

print('-' * 62)
print(f'Mejor Val Accuracy: {best_val_acc:.2f}%')

```

Entrenando ResNet-50 en cuda durante 10 epocas...

```
-----
Ep   | Train Loss | Train Acc | Val Loss | Val Acc
-----

1/10 |    0.4155 |    88.08% |    0.1341 |    95.62% <- mejor
```

```
Exception ignored in: <function _MultiProcessingDataLoaderIter.__del__ at 0x7adef85b3100>
Exception ignored in: Traceback (most recent call last):
<function _MultiProcessingDataLoaderIter.__del__ at 0x7adef85b3100>
  File "/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py", line 1709, in __del__
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py", line 1709, in __del__
self._shutdown_workers()
self._shutdown_workers() File "/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py", line 1692, in _shut
  File "/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py", line 1692, in _shutdown_workers
if w.is_alive():
  if w.is_alive():
    ^ ^ ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in is_alive
    ^
assert self._parent_pid == os.getpid(), 'can only test a child process' File "/usr/lib/python3.12/multiprocessing/process.p
  assert self._parent_pid == os.getpid(), 'can only test a child process'
    ^ ^ ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionErrorAssertionError: : can only test a child processcan only test a child process
```

```
2/10 |    0.1116 |    96.36% |    0.0845 |    97.26% <- mejor
```

```
3/10 |    0.0783 |    97.43% |    0.0742 |    97.57% <- mejor
```

Train: 63% 763/1213 [03:12<01:39, 4.54it/s]

```
-----
KeyboardInterrupt Traceback (most recent call last)
/tmp/ipykernel_1791/856367717.py in <cell line: 0>()
    21
    22 for epoch in range(1, EPOCHS + 1):
--> 23     train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer, device)
    24     val_loss, val_acc = eval_epoch(model, val_loader, criterion, device)
    25

/tmp/ipykernel_1791/1863505502.py in train_epoch(model, loader, criterion, optimizer, device)
    13     optimizer.step()
    14
--> 15     running_loss += loss.item()
    16     _, predicted = outputs.max(1)
    17     total += labels.size(0)
```

KeyboardInterrupt:

```
checkpoint_path = '/content/checkpoints/checkpoint_epoch_8.pth' ##PONER DONDE SE LES QUEDO
```

```
if os.path.exists(checkpoint_path):
    checkpoint = torch.load(checkpoint_path)

    # Restaurar estados
    model.load_state_dict(checkpoint['model_state_dict'])
    optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
    scheduler.load_state_dict(checkpoint['scheduler_state_dict'])
    history = checkpoint['history']
    best_val_acc = checkpoint['best_val_acc']
    start_epoch = checkpoint['epoch']

    print(f"> Checkpoint cargado con éxito. Reanudando desde la época {start_epoch + 1}")
    # Nota: Si reanudas, tu bucle for debería empezar en range(start_epoch + 1, EPOCHS + 1)
```

```

-----
RuntimeError                                Traceback (most recent call last)
/tmp/ipykernel_15994/3320314706.py in <cell line: 0>()
      2
      3 if os.path.exists(checkpoint_path):
----> 4     checkpoint = torch.load(checkpoint_path)
      5
      6     # Restaurar estados

----- 1 frames -----
/usr/local/lib/python3.12/dist-packages/torch/serialization.py in __init__(self, name_or_buffer)
    805 class _open_zipfile_reader(_opener[torch._C.PyTorchFileReader]):
    806     def __init__(self, name_or_buffer: str | IO[bytes]) -> None:
--> 807         super().__init__(torch._C.PyTorchFileReader(name_or_buffer))
    808
    809

RuntimeError: PytorchStreamReader failed reading zip archive: failed finding central directory. This is an internal miniz
error. If you are seeing this error, there is a high likelihood that your checkpoint file is corrupted. This can happen if
the checkpoint was not saved properly, was transferred incorrectly, or the file was modified after saving.

```

6. Analisis Preliminar de Resultados

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

eps = range(1, len(history['train_loss']) + 1)

ax1.plot(eps, history['train_loss'], 'b-o', label='Train', linewidth=2, markersize=6)
ax1.plot(eps, history['val_loss'], 'r-s', label='Validacion', linewidth=2, markersize=6)
ax1.set_title('Curva de Perdida (Loss)', fontsize=13, fontweight='bold')
ax1.set_xlabel('Epoca'); ax1.set_ylabel('Cross-Entropy Loss')
ax1.legend(fontsize=11); ax1.grid(True, alpha=0.3)

ax2.plot(eps, history['train_acc'], 'b-o', label='Train', linewidth=2, markersize=6)
ax2.plot(eps, history['val_acc'], 'r-s', label='Validacion', linewidth=2, markersize=6)
ax2.set_title('Curva de Precision (Accuracy)', fontsize=13, fontweight='bold')
ax2.set_xlabel('Epoca'); ax2.set_ylabel('Accuracy (%)')
ax2.legend(fontsize=11); ax2.grid(True, alpha=0.3)

plt.suptitle('Historial de Entrenamiento - ResNet-50 PlantDoc', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('/content/training_curves.png', dpi=150, bbox_inches='tight')
plt.show()

print(f'Mejor accuracy en validacion: {max(history["val_acc"]):.2f}%')
print(f'Epoca optima: {history["val_acc"].index(max(history["val_acc"])) + 1}')

```

```
best_model_path = '/content/checkpoint_epoch_8.pth'
```

```

import torch

# Cargar el mejor modelo guardado
checkpoint = torch.load(best_model_path, weights_only=False)
model.load_state_dict(checkpoint['model_state_dict'])
model.eval()

all_preds = []
all_labels = []

with torch.no_grad():
    for images, labels in tqdm(test_loader, desc='Evaluando test set'):
        images = images.to(device)
        outputs = model(images)
        _, preds = outputs.max(1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.numpy())

test_acc = 100.0 * sum(p == 1 for p, l in zip(all_preds, all_labels)) / len(all_labels)
print(f'\n=== Accuracy Final en Test Set: {test_acc:.2f}% ===')

```

```

RuntimeError                                Traceback (most recent call last)
/tmp/ipykernel_15994/2585017671.py in <cell line: 0>()
      2
      3 # Cargar el mejor modelo guardado
----> 4 checkpoint = torch.load(best_model_path, weights_only=False)
      5 model.load_state_dict(checkpoint['model_state_dict'])
      6 model.eval()

```

1 frames

```

/usr/local/lib/python3.12/dist-packages/torch/serialization.py in __init__(self, name_or_buffer)
    805 class _open_zipfile_reader(_opener[torch._C.PyTorchFileReader]):
    806     def __init__(self, name_or_buffer: str | IO[bytes]) -> None:
--> 807         super().__init__(torch._C.PyTorchFileReader(name_or_buffer))
    808
    809

```

RuntimeError: PytorchStreamReader failed reading zip archive: failed finding central directory. This is an internal miniz error. If you are seeing this error, there is a high likelihood that your checkpoint file is corrupted. This can happen if the checkpoint was not saved properly, was transferred incorrectly, or the file was modified after saving.

```

report_dict = classification_report(
    all_labels, all_preds,
    target_names=class_names,
    output_dict=True,
    zero_division=0
)

df_report = pd.DataFrame(report_dict).T
df_report = df_report.drop(
    labels=[c for c in ['accuracy', 'macro avg', 'weighted avg'] if c in df_report.index],
    errors='ignore'
)
df_report = df_report[['precision', 'recall', 'f1-score', 'support']].round(3)
df_report['support'] = df_report['support'].astype(int)
df_report = df_report.sort_values('f1-score', ascending=False)

print('Top 15 clases por F1-Score:')
display(df_report.head(15))

print(f'\nBottom 5 clases por F1-Score (mas dificiles):')
display(df_report.tail(5))

print(f'\nMacro F1-Score: {report_dict["macro avg"]["f1-score"]:.4f}')
print(f'Weighted F1-Score: {report_dict["weighted avg"]["f1-score"]:.4f}')

```

```

cm = confusion_matrix(all_labels, all_preds)

# Top 15 clases con mas imagenes en test
class_counts_test = np.bincount(all_labels, minlength=len(class_names))
top15_idx = np.argsort(class_counts_test)[-15:]
cm_top = cm[np.ix_(top15_idx, top15_idx)]
top15_names = [class_names[i][22] for i in top15_idx]

# Version normalizada (tasa de acierto por clase)
cm_norm = cm_top.astype(float) / (cm_top.sum(axis=1, keepdims=True) + 1e-8)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(22, 10))

sns.heatmap(cm_top, annot=True, fmt='d', cmap='Blues',
            xticklabels=top15_names, yticklabels=top15_names, ax=ax1)
ax1.set_title('Matriz de Confusion - Conteos absolutos', fontsize=12, fontweight='bold')
ax1.set_xlabel('Prediccion'); ax1.set_ylabel('Real')
ax1.tick_params(axis='x', rotation=45)

sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Greens',
            xticklabels=top15_names, yticklabels=top15_names, ax=ax2)
ax2.set_title('Matriz de Confusion - Normalizada por fila', fontsize=12, fontweight='bold')
ax2.set_xlabel('Prediccion'); ax2.set_ylabel('Real')
ax2.tick_params(axis='x', rotation=45)

plt.suptitle('Top 15 Clases mas frecuentes en Test Set', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('/content/confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()

```

```

model.eval()

inv_norm = transforms.Normalize(
    mean=[-0.485/0.229, -0.456/0.224, -0.406/0.225],

```

```

std=[1/0.229, 1/0.224, 1/0.225]
)

fig, axes = plt.subplots(3, 4, figsize=(18, 13))
axes = axes.flatten()

count = 0
with torch.no_grad():
    for images, labels in test_loader:
        images_gpu = images.to(device)
        outputs = model(images_gpu)
        probs = torch.softmax(outputs, dim=1)
        confs, preds = probs.max(1)

        for j in range(images.size(0)):
            if count >= 12:
                break
            img = inv_norm(images[j]).permute(1, 2, 0).numpy()
            img = np.clip(img, 0, 1)
            true = class_names[labels[j]]
            pred = class_names[preds[j].item()]
            conf = confs[j].item()
            ok = (labels[j].item() == preds[j].item())

            axes[count].imshow(img)
            title_text = f'Real: {true[:22]} ' + '\n' + f'Pred: {pred[:22]} ' + '\n' + f'Conf: {conf:.1%}'
            axes[count].set_title(title_text, fontsize=8,
                                color='darkgreen' if ok else 'crimson')
            axes[count].axis('off')
            count += 1

        if count >= 12:
            break

correct_patch = mpatches.Patch(color='darkgreen', label='Prediccion Correcta')
wrong_patch = mpatches.Patch(color='crimson', label='Prediccion Incorrecta')
fig.legend(handles=[correct_patch, wrong_patch],
           loc='lower center', ncol=2, fontsize=11, framealpha=0.9)

plt.suptitle('Predicciones del Modelo ResNet-50 sobre Test Set',
             fontsize=14, fontweight='bold')
plt.tight_layout(rect=[0, 0.04, 1, 1])
plt.savefig('/content/sample_predictions.png', dpi=150, bbox_inches='tight')
plt.show()

```

7. Conclusiones y Proximos Pasos

Observaciones tecnicas

- **Transfer Learning efectivo:** Los pesos preentrenados en ImageNet permiten que ResNet-50 converja rapido incluso con solo 10 epocas de fine-tuning sobre las capas superiores.
- **Skip Connections:** Las conexiones residuales evitan el vanishing gradient y permiten entrenar la red profunda de forma estable, sin degradacion del gradiente en las primeras capas.
- **Data Augmentation:** Las transformaciones (flip, rotacion, color jitter) aumentan la diversidad del conjunto de entrenamiento y reducen el sobreajuste.
- **Matriz de confusion:** Permite identificar que patologias el modelo confunde mas (tipicamente enfermedades de la misma especie con sintomas visuales similares).

Proximos pasos (Entrega Final)

1. **Fine-tuning progresivo:** descongelar `layer3` y `layer2` en etapas posteriores con lr mas baja
2. **Mas epocas:** extender a 30-50 epocas para convergencia completa
3. **Augmentation avanzado:** explorar Mixup, CutMix o AutoAugment
4. **Analisis de errores:** estudiar los patrones en imagenes mal clasificadas
5. **Comparacion:** evaluar EfficientNet-B4 como alternativa mas eficiente
6. **Exportacion:** guardar en formato ONNX para inferencia en produccion

✓ 8. Probar con Imágenes Propias

Ejecuta la siguiente celda para definir la función de predicción y luego usa el bloque de abajo para probar con una imagen local.

```

def predecir_imagen_externa(ruta_imagen, model, transform, class_names, device, criterion=None):
    """Carga, procesa y predice una imagen mostrando el Top 10 de clases más probables."""

```

```

if not os.path.exists(ruta_imagen):
    print(f'Error: No se encuentra el archivo en {ruta_imagen}')
    return

# 1. Cargar imagen (aquí puedes mantener tu lógica de OpenCV/PIL que prefieras)
img_pil = Image.open(ruta_imagen).convert('RGB')
img_t = transform(img_pil).unsqueeze(0).to(device)

model.eval()
with torch.no_grad():
    output = model(img_t)
    prob = torch.softmax(output, dim=1) # Vector de probabilidades para todas las clases

    # En lugar de solo el max, extraemos los 10 valores más altos
    # torch.topk nos da los valores (confianzas) y sus respectivos índices
    top_conf, top_idx = torch.topk(prob, min(10, len(class_names)), dim=1)

    # Pasamos a CPU y a arrays de numpy para manipularlos fácilmente
    top_conf = top_conf.squeeze().cpu().numpy()
    top_idx = top_idx.squeeze().cpu().numpy()

    # La predicción principal sigue siendo el puesto #1
    pred_principal = top_idx[0]
    conf_principal = top_conf[0]

    loss_val = None
    if criterion is not None:
        # El loss se calcula con la clase ganadora
        target_tensor = torch.tensor([pred_principal]).to(device)
        loss_val = criterion(output, target_tensor).item()

# 2. Visualización de la imagen analizada
plt.figure(figsize=(5, 5))
plt.imshow(img_pil)
plt.title(f'Top 1: {class_names[pred_principal]}\nConfianza: {conf_principal:.2%}')
plt.axis('off')
plt.show()

# 3. Imprimir el ranking interno de la imagen
print("\n" + "="*60)
print(f" 🖼️ RANKING DE PROBABILIDAD PARA: {os.path.basename(ruta_imagen)}")
print("="*60)
for i in range(len(top_idx)):
    print(f" Puesto {i+1:2d} | {class_names[top_idx[i]]:<45} | {top_conf[i]:.2%}")
print("="*60 + "\n")

return class_names[pred_principal], conf_principal, loss_val

```

```
!pip install -q gradio
```

```

from google.colab import files

print("Sube una o varias imágenes de hojas para analizar:")
uploaded = files.upload()

# Lista para almacenar los resultados de todas las imágenes
resultados_totales = []

# 1. Procesar todas las imágenes subidas y guardar sus métricas
for fn in uploaded.keys():
    clase_detectada, confianza, loss_val = predecir_imagen_externa(
        fn, model, transform_eval, class_names, device, criterion
    )

    # Guardamos los datos en un diccionario para poder ordenarlos fácilmente
    resultados_totales.append({
        'archivo': fn,
        'clase': clase_detectada,
        'confianza': confianza,
        'loss': loss_val
    })

# 2. Ordenar la lista por 'confianza' de mayor a menor (reverse=True)
resultados_ordenados = sorted(resultados_totales, key=lambda x: x['confianza'], reverse=True)

# 3. Imprimir el Ranking Top 10 con un formato limpio y elegante
print("\n" + "="*70)
print(" 🏆 RANKING TOP 10: PREDICCIONES DE MAYOR A MENOR CONFIANZA")
print("="*70)

# Tomamos un máximo de 10 elementos de la lista ordenada

```



```

for puesto, res in enumerate(resultados_ordenados[:10], start=1):
    loss_str = f" | Loss: {res['loss']:.4f}" if res['loss'] is not None else ""

    print(f"{puesto:2d}. Archivo: {res['archivo']}\n"
          f"      Clase Predicha: {res['clase']}\n"
          f"      Confianza:      {res['confianza']:.2%}{loss_str}")
    print("-" * 70)

if len(resultados_ordenados) == 0:
    print("No se subió ninguna imagen válida.")

```

Sube una o varias imágenes de hojas para analizar:

Ningún archivo seleccionado Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipykernel_1791/1351178510.py in <cell line: 0>()
      2
      3 print("Sube una o varias imágenes de hojas para analizar:")
----> 4 uploaded = files.upload()
      5
      6 # Lista para almacenar los resultados de todas las imágenes

```

```

----- 3 frames -----
/usr/local/lib/python3.12/dist-packages/google/colab/_message.py in read_reply_from_input(message_id, timeout_sec)
    94     reply = _read_next_input_message()
    95     if reply == _NOT_READY or not isinstance(reply, dict):
----> 96         time.sleep(0.025)
    97         continue
    98     if (

```

KeyboardInterrupt:

!pip install ultralytics

Collecting ultralytics

Downloading ultralytics-8.4.60-py3-none-any.whl.metadata (41 kB)

41.2/41.2 kB 2.7 MB/s eta 0:00:00

```

Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.2)
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)
Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (4.13.0)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.16.3)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.11.0+cu128)
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (0.26.0+cu128)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (5.9.5)
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.35.2)
Collecting ultralytics-thop>=2.0.18 (from ultralytics)
  Downloading ultralytics-thop-2.0.19-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultral)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultral)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultral)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultral)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultral)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultral)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultral)
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.12/dist-packages (from polars>=0.20.0->ultral)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultral)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultral)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultral)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultral)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral) (3.29)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral) (69.5.1)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral) (1.13.3)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral) (3.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral) (2024.10.1)
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,c]
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral)
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral)
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultral) (3.6.0)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,c]
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,c]
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,c]
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,c]
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,c]
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,c]
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,c]
Requirement already satisfied: nvidia-cusparselt-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,c]
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,c]
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,c]

```

Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cuda-nvtx-cu12]==12.8.90) 12.8.90.0
 Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings<13,>=12) 1.1.0
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) 1.16.0
 Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.13.1) 1.3.0
 Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.13.1) 2.1.5

```
print(class_names)
```

```
['Apple__Apple_scab', 'Apple__Black_rot', 'Apple__Cedar_apple_rust', 'Apple__healthy', 'Background_without_leaves', 'Blu
```

```
import os
import torch
import torch.nn as nn
from torchvision import models, transforms
from ultralytics import YOLO
import gradio as gr
import cv2
import numpy as np
from PIL import Image

# =====
# 1. DETECCIÓN AUTOMÁTICA DE CLASES DESDE EL DIRECTORIO
# =====
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Usando dispositivo: {device}")

# Tu ruta original para escanear las carpetas
dataset_path = '/content/dataset_plantas/Plant_leave_diseases_dataset_without_augmentation'

if os.path.exists(dataset_path):
    class_names = sorted([
        c for c in os.listdir(dataset_path)
        if os.path.isdir(os.path.join(dataset_path, c))
    ])
    num_clases_detectadas = len(class_names)
    print(f"✅ Éxito: Se detectaron automáticamente {num_clases_detectadas} clases.")

# Transformaciones de evaluación
transform_eval = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# =====
# 2. CARGAR DETECTOR (YOLOv8) Y CLASIFICADOR (ResNet-50)
# =====
# Fijamos 'task=detect' para que no arroje advertencias en la consola
detector_hojas = YOLO('yolov8n.pt', task='detect')

# Define el directorio de checkpoints
checkpoint_dir = '/content/checkpoints'
# Asegura que path_tus_pesos apunte al archivo correcto en el directorio de checkpoints
path_tus_pesos = os.path.join(checkpoint_dir, 'plantdoc_resnet50_best.pth')

# Instanciar arquitectura ResNet-50
modelo_resnet = models.resnet50(weights=None)
num_fts = modelo_resnet.fc.in_features

# Replicamos el secuencial idéntico de tu entrenamiento usando el conteo automático
modelo_resnet.fc = nn.Sequential(
    nn.Identity(), # Posición 0
    nn.Linear(num_fts, num_clases_detectadas) # Posición 1: Encaja con las 39 clases de tus pesos
)

# Carga de los pesos entrenados
if os.path.exists(path_tus_pesos):
    checkpoint = torch.load(path_tus_pesos, map_location=device)
    modelo_resnet.load_state_dict(checkpoint['model_state_dict'], strict=True)
    print("¡Tus pesos de ResNet-50 (.pth) han sido vinculados correctamente con sus nombres reales!")
else:
    print(f"❌ Error crítico: Sube tu archivo '{path_tus_pesos}' a Colab para que el diagnóstico sea real.")

modelo_resnet.to(device)
modelo_resnet.eval()

# =====
# 3. LÓGICA DEL PROCESAMIENTO HÍBRIDO (DETECCIÓN + CLASIFICACIÓN)
# =====
def segmentar_y_clasificar_con_pth(img_pil):
```

```

if img_pil is None:
    return None, "Por favor, sube una imagen válida."

img_cv = cv2.cvtColor(np.array(img_pil), cv2.COLOR_RGB2BGR)
img_out = img_cv.copy()

# YOLO busca cualquier parche orgánico con umbral bajo para máxima sensibilidad
results = detector_hojas(img_cv, conf=0.05, verbose=False)

reporte_lineas = []
contador_hojas = 0

for box in results[0].boxes:
    x1, y1, x2, y2 = map(int, box.xyxy[0])

    recorte_bgr = img_cv[y1:y2, x1:x2]
    if recorte_bgr.size == 0:
        continue

    recorte_pil = Image.fromarray(cv2.cvtColor(recorte_bgr, cv2.COLOR_BGR2RGB))
    img_t = transform_eval(recorte_pil).unsqueeze(0).to(device)

    with torch.no_grad():
        output = modelo_resnet(img_t)
        prob = torch.softmax(output, dim=1)
        conf, pred = torch.max(prob, 1)
        clase_resnet = class_names[pred.item()]

    # Filtrar si clasifica el recorte como fondo
    if "Background" in clase_resnet:
        continue

    contador_hojas += 1

    # Dibujar rectángulos y etiquetas reales de tus carpetas
    cv2.rectangle(img_out, (x1, y1), (x2, y2), (0, 255, 0), 3)

    # Limpiar los nombres de las carpetas para que se lean bien en el recuadro gráfico
    nombre_legible = clase_resnet.split("___")[-1].replace("_", " ") if "___" in clase_resnet else clase_resnet.replace(
        label = f"H{contador_hojas}: {nombre_legible} ({conf.item():.1%})"
    cv2.putText(img_out, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

    reporte_lineas.append(f"🌿 **Hoja #{contador_hojas}**: `{clase_resnet}` con confianza del **{conf.item():.2%**}")

if contador_hojas == 0:
    texto_out = "❌ No se lograron aislar hojas individuales en esta toma. Prueba con una foto más cercana o con mejor
else:
    texto_out = f"### 📋 Diagnóstico usando tu ResNet-50 (.pth):\n" + "\n".join(reporte_lineas)

img_final = Image.fromarray(cv2.cvtColor(img_out, cv2.COLOR_BGR2RGB))
return img_final, texto_out

# =====
# 4. CONFIGURACIÓN Y LANZAMIENTO DE GRADIO
# =====
with gr.Blocks(theme=gr.themes.Soft()) as demo_hibrida:
    gr.Markdown("# 🌿 Escáner Híbrido Avanzado (YOLOv8 + Tu ResNet-50 .pth)")
    gr.Markdown("Esta interfaz utiliza YOLOv8 para localizar hojas en entornos complejos y aplica **tu archivo .pth** mapea:

    with gr.Row():
        with gr.Column():
            input_box = gr.Image(type="pil", label="Foto del cultivo (Mundo Real)")
            btn_run = gr.Button("🔍 Escanear con mi Modelo .pth", variant="primary")
        with gr.Column():
            output_image = gr.Image(type="pil", label="Hojas localizadas y diagnosticadas")
            output_text = gr.Markdown(label="Reporte Clínico")

    btn_run.click(fn=segmentar_y_clasificar_con_pth, inputs=input_box, outputs=[output_image, output_text])

demo_hibrida.launch(share=True)

```

Creating new Ultralytics Settings v0.0.6 file 

View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'

Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics.com>

Usando dispositivo: cuda

 Éxito: Se detectaron automáticamente 39 clases.

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt> to 'yolov8n.pt': 100% ————— 6.2 MB

¡Tus pesos de ResNet-50 (.pth) han sido vinculados correctamente con sus nombres reales!

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()

* Running on public URL: <https://0a36eafec1bba40f94.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the notebook.



No interface is running right now

Sección MODELADO — Experimentación y Ablation Study

PlantDoc · ResNet-50 · Clasificación de 39 Patologías Vegetales

Este notebook **debe ejecutarse luego del notebook principal** (secciones 1–6), ya que reutiliza las variables `train_loader`, `val_loader`, `test_loader`, `class_names` y `device` que fueron definidas allí.

Mapa de Experimentos

#	Variable	Valores probados	Fijo
Exp 1	Learning Rate	<code>1e-2</code> , <code>1e-3</code> , <code>5e-4</code> , <code>1e-4</code>	Adam, BS=32, layer4+fc
Exp 2	Batch Size	<code>16</code> , <code>32</code> , <code>64</code>	Adam, lr=1e-3, layer4+fc
Exp 3	Capas descongeladas	<code>fc_only</code> , <code>layer4+fc</code> , <code>layer3+4+fc</code> , <code>layer2+3+4+fc</code>	Adam, lr=1e-3, BS=32
Exp 4	Función de activación	<code>none</code> , <code>relu</code> , <code>tanh</code> , <code>sigmoid</code> , <code>gelu</code> , <code>leaky_relu</code>	Adam, lr=1e-3, BS=32, layer4+fc
Exp 5	Optimizador	<code>Adam</code> , <code>AdamW</code> , <code>SGD</code> , <code>SGD+Nesterov</code> , <code>RMSprop</code>	lr=1e-3, BS=32, layer4+fc

Nota de tiempo: Cada experimento entrena **5 épocas** sobre los mismos splits 70/15/15 del notebook base.

Métricas comparadas: `val_acc`, `val_loss`, `test_acc`, `tiempo_entrenamiento`, `best_epoch`.

Setup — Imports y Verificación de Prerequisitos

```
import os, time, gc, copy, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms, models
from sklearn.metrics import classification_report
from tqdm.auto import tqdm
```

```
warnings.filterwarnings('ignore')

# — Reproducibilidad global —————
SEED = 42
import random
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

# — Verificar variables del notebook base ———
required_vars = ['train_loader', 'val_loader', 'test_loader', 'class_names', 'device']
missing = [v for v in required_vars if v not in dir()]
if missing:
    raise EnvironmentError(
        f"❌ Faltan las variables: {missing}\n"
        "Ejecuta primero las secciones 1-5 del notebook principal (COLAB_FINAL.ipynb)"
    )

NUM_CLASSES = len(class_names)
EXP_EPOCHS = 5 # épocas por experimento (balance velocidad/información)
BATCH_SIZE = 32 # batch size base para la mayoría de experimentos

print(f"✅ Todas las variables del notebook base están disponibles")
print(f" • Clases : {NUM_CLASSES}")
print(f" • Dispositivo : {device}")
print(f" • Épocas / exp : {EXP_EPOCHS}")
print(f" • Batch base : {BATCH_SIZE}")
print(f" • Train batches : {len(train_loader)}")
print(f" • Val batches : {len(val_loader)}")
print(f" • Test batches : {len(test_loader)}")
```

```
✅ Todas las variables del notebook base están disponibles
• Clases : 39
• Dispositivo : cuda
• Épocas / exp : 5
• Batch base : 32
• Train batches : 1213
• Val batches : 260
• Test batches : 260
```

✂ Funciones Auxiliares

Todas las funciones de apoyo que comparten los 5 experimentos:

- `build_model()` → construye ResNet-50 con la cabeza y el descongelado configurables
- `build_optimizer()` → instancia el optimizador con learning rates diferenciadas (backbone vs. cabeza)
- `run_experiment()` → ciclo completo train/val/test con registro de métricas
- `plot_exp_curves()` → curvas de pérdida y accuracy por experimento
- `summary_table()` → tabla comparativa al final de cada bloque

```
# =====
# build_model
# =====
def build_model(
    num_classes,
    unfreeze_config='layer4+fc', # qué capas se descongelan
    head_activation=None, # None | 'relu' | 'tanh' | 'sigmoid' | 'gelu' | 'leaky_relu'
    hidden_dim=512,
    dropout=0.5
):
    """
    Construye ResNet-50 con pesos ImageNet.

    Estrategias de descongelado disponibles:
    'fc_only' → solo la nueva cabeza
    'layer4+fc' → layer4 + cabeza (configuración base del proyecto)
    'layer3+4+fc' → layer3, layer4 + cabeza
    'layer2+3+4+fc' → layer2, layer3, layer4 + cabeza

    Si head_activation es None se usa la cabeza simple:
    Dropout(p) → Linear(2048 → num_classes)

    Si head_activation está especificada, se añade una capa oculta:
```

```

Dropout(p) → Linear(2048 → hidden_dim) → ACT → Dropout(p/2) → Linear(hidden_dim → num_classes)
"""
try:
    model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
except AttributeError:
    model = models.resnet50(pretrained=True)

# — 1. Congelar todo —————
for p in model.parameters():
    p.requires_grad = False

# — 2. Descongelar selectivamente —————
unfreeze_map = {
    'fc_only'      : [],
    'layer4+fc'    : ['layer4'],
    'layer3+4+fc'  : ['layer3', 'layer4'],
    'layer2+3+4+fc': ['layer2', 'layer3', 'layer4'],
}
for layer_name in unfreeze_map.get(unfreeze_config, ['layer4']):
    for p in getattr(model, layer_name).parameters():
        p.requires_grad = True

# — 3. Cabeza clasificadora —————
nf = model.fc.in_features # 2048 en ResNet-50

act_map = {
    'relu'      : nn.ReLU(inplace=True),
    'tanh'      : nn.Tanh(),
    'sigmoid'   : nn.Sigmoid(),
    'gelu'      : nn.GELU(),
    'leaky_relu': nn.LeakyReLU(negative_slope=0.1, inplace=True),
}

if head_activation is None:
    model.fc = nn.Sequential(
        nn.Dropout(dropout),
        nn.Linear(nf, num_classes)
    )
else:
    model.fc = nn.Sequential(
        nn.Dropout(dropout),
        nn.Linear(nf, hidden_dim),
        act_map[head_activation],
        nn.Dropout(dropout * 0.5),
        nn.Linear(hidden_dim, num_classes)
    )

total      = sum(p.numel() for p in model.parameters())
trainable  = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"  · Parámetros entrenables: {trainable:,} / {total:,}  ({trainable/total:.1%})")
print(f"  · Cabeza: {model.fc}")

return model.to(device)

# =====
# build_optimizer
# =====
def build_optimizer(model, optimizer_name, lr_fc, lr_backbone=None):
    """
    Construye el optimizador con tasas diferenciadas:
    lr_backbone = lr_fc / 10 por defecto (backbone ya preentrenado → lr menor)
    """
    if lr_backbone is None:
        lr_backbone = lr_fc / 10.0

    fc_ids      = {id(p) for p in model.fc.parameters()}
    backbone_p  = [p for p in model.parameters()
                    if p.requires_grad and id(p) not in fc_ids]
    fc_p        = list(model.fc.parameters())

    groups = []
    if backbone_p:
        groups.append({'params': backbone_p, 'lr': lr_backbone})
    groups.append({'params': fc_p, 'lr': lr_fc})

    opts = {
        'Adam'      : lambda: optim.Adam(groups, weight_decay=1e-4),
        'AdamW'     : lambda: optim.AdamW(groups, weight_decay=1e-2),
        'SGD'       : lambda: optim.SGD(groups, momentum=0.0, weight_decay=1e-4),
        'SGD_Nesterov': lambda: optim.SGD(groups, momentum=0.9, nesterov=True, weight_decay=1e-4),
        'RMSprop'   : lambda: optim.RMSprop(groups, momentum=0.9, weight_decay=1e-4),
    }

```

```

    }
    return opts[optimizer_name]()

# =====
# run_experiment
# =====
def run_experiment(config, train_loader, val_loader, test_loader, epochs=EXP_EPOCHS, verbose=True):
    """
    Ejecuta un experimento completo y devuelve un diccionario con:
        history, test_acc, best_val_acc, best_val_loss, training_time, config
    """
    torch.manual_seed(SEED)

    model = build_model(
        num_classes = NUM_CLASSES,
        unfreeze_config= config.get('unfreeze_config', 'layer4+fc'),
        head_activation= config.get('head_activation', None),
        dropout = config.get('dropout', 0.5),
    )

    optimizer = build_optimizer(
        model,
        optimizer_name = config.get('optimizer_name', 'Adam'),
        lr_fc = config.get('lr_fc', 1e-3),
        lr_backbone = config.get('lr_backbone', None),
    )

    criterion = nn.CrossEntropyLoss(label_smoothing=0.05)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode='min', factor=0.5, patience=2
    )

    history = {k: [] for k in ('train_loss', 'val_loss', 'train_acc', 'val_acc')}
    t0 = time.time()

    for epoch in range(1, epochs + 1):
        # — Train —————
        model.train()
        r_loss = c = n = 0
        for imgs, lbls in tqdm(train_loader, desc=f'Ep {epoch}/{epochs} Train', leave=False):
            imgs, lbls = imgs.to(device), lbls.to(device)
            optimizer.zero_grad()
            out = model(imgs)
            loss = criterion(out, lbls)
            loss.backward()
            optimizer.step()
            r_loss += loss.item(); _, p = out.max(1)
            n += lbls.size(0); c += p.eq(lbls).sum().item()
        t1 = r_loss / len(train_loader); ta = 100.0 * c / n

        # — Validación —————
        model.eval(); r_loss = c = n = 0
        with torch.no_grad():
            for imgs, lbls in tqdm(val_loader, desc=f'Ep {epoch}/{epochs} Val ', leave=False):
                imgs, lbls = imgs.to(device), lbls.to(device)
                out = model(imgs); loss = criterion(out, lbls)
                r_loss += loss.item(); _, p = out.max(1)
                n += lbls.size(0); c += p.eq(lbls).sum().item()
            vl = r_loss / len(val_loader); va = 100.0 * c / n
            scheduler.step(vl)

        history['train_loss'].append(t1); history['val_loss'].append(vl)
        history['train_acc'].append(ta); history['val_acc'].append(va)

        if verbose:
            print(f" Ep {epoch:02d}/{epochs} | "
                  f"Train loss={t1:.4f} acc={ta:6.2f}% | "
                  f"Val loss={vl:.4f} acc={va:6.2f}%")

    # — Test final —————
    model.eval(); c = n = 0
    with torch.no_grad():
        for imgs, lbls in tqdm(test_loader, desc='Test', leave=False):
            imgs, lbls = imgs.to(device), lbls.to(device)
            _, p = model(imgs).max(1)
            n += lbls.size(0); c += p.eq(lbls).sum().item()
    test_acc = 100.0 * c / n
    elapsed = time.time() - t0

    print(f" ✓ Test Acc: {test_acc:.2f}% Tiempo: {elapsed/60:.1f} min")

```



```

result = {
    'history'      : history,
    'test_acc'     : test_acc,
    'best_val_acc': max(history['val_acc']),
    'best_val_loss': min(history['val_loss']),
    'best_epoch'   : int(np.argmax(history['val_acc'])) + 1,
    'training_time': elapsed,
    'config'       : copy.deepcopy(config),
}

# Liberar GPU
del model
torch.cuda.empty_cache(); gc.collect()
return result

# =====
# Utilidades de visualización
# =====
PALETTE = plt.cm.tab10.colors # colores consistentes entre plots

def plot_exp_curves(results_dict, title='', metric_pair=('val_loss', 'val_acc')):
    """
    Curvas de loss y accuracy para un grupo de experimentos.
    results_dict: {label: result_dict}
    """
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
    eps = range(1, EXP_EPOCHS + 1)

    for i, (label, res) in enumerate(results_dict.items()):
        clr = PALETTE[i % len(PALETTE)]
        ax1.plot(eps, res['history']['val_loss'], '-o', color=clr, label=label, lw=2, ms=5)
        ax2.plot(eps, res['history']['val_acc'], '-o', color=clr, label=label, lw=2, ms=5)

    for ax, ylab, ttl in zip(
        [ax1, ax2],
        ['Val Loss (Cross-Entropy)', 'Val Accuracy (%)'],
        [f'Val Loss - {title}', f'Val Accuracy - {title}']
    ):
        ax.set_xlabel('Época'); ax.set_ylabel(ylab); ax.set_title(ttl, fontweight='bold')
        ax.legend(fontsize=9); ax.grid(True, alpha=0.3)
        ax.set_xticks(eps)

    plt.tight_layout()
    plt.savefig(f'/content/exp_{title.lower().replace(" ", "_")}.png', dpi=130, bbox_inches='tight')
    plt.show()

def summary_table(results_dict, exp_name=''):
    """
    Tabla comparativa de métricas finales.
    """
    rows = []
    for label, res in results_dict.items():
        rows.append({
            'Configuración' : label,
            'Best Val Acc (%)' : f"{res['best_val_acc']:.2f}",
            'Best Val Loss'   : f"{res['best_val_loss']:.4f}",
            'Test Acc (%)'    : f"{res['test_acc']:.2f}",
            'Best Epoch'      : res['best_epoch'],
            'Tiempo (min)'     : f"{res['training_time']/60:.1f}",
        })

    df = pd.DataFrame(rows).set_index('Configuración')

    # Destacar la mejor fila
    best_idx = df['Test Acc (%)'].astype(float).idxmax()
    print(f"\n{'='*70}")
    print(f" 📊 RESUMEN - {exp_name}")
    print(f"{'='*70}")
    print(df.to_string())
    print(f"\n 🏆 MEJOR CONFIGURACIÓN: '{best_idx}' "
          f"(Test Acc = {df.loc[best_idx, 'Test Acc (%)']:%})")
    print(f"{'='*70}\n")
    return df

print("✅ Funciones auxiliares definidas correctamente")

```

✅ Funciones auxiliares definidas correctamente

Experimento 1 — Learning Rate

Hipótesis: Una LR demasiado alta causa inestabilidad en el fine-tuning de `layer4` (ya preentrenada).

Una LR muy baja ralentiza la convergencia de la cabeza nueva. Se espera que $lr \in [5e-4, 1e-3]$ sea el rango óptimo.

Configuración	lr_fc	lr_backbone (layer4)
LR = 1e-2	0.010	0.001
LR = 1e-3 ★ baseline	0.001	0.0001
LR = 5e-4	0.0005	0.00005
LR = 1e-4	0.0001	0.00001

Variables fijas: Adam · BatchSize=32 · `layer4+fc` descongeladas · 5 épocas

```
print("=" * 60)
print(" EXPERIMENTO 1: LEARNING RATE")
print("=" * 60)

lr_configs = {
    'LR = 1e-2' : {'lr_fc': 1e-2, 'optimizer_name': 'Adam', 'unfreeze_config': 'layer4+fc'},
    'LR = 1e-3' : {'lr_fc': 1e-3, 'optimizer_name': 'Adam', 'unfreeze_config': 'layer4+fc'},
    'LR = 5e-4' : {'lr_fc': 5e-4, 'optimizer_name': 'Adam', 'unfreeze_config': 'layer4+fc'},
    'LR = 1e-4' : {'lr_fc': 1e-4, 'optimizer_name': 'Adam', 'unfreeze_config': 'layer4+fc'},
}

results_lr = {}
for label, cfg in lr_configs.items():
    print(f"\n► Probandando {label} ...")
    results_lr[label] = run_experiment(
        config      = cfg,
        train_loader = train_loader,
        val_loader   = val_loader,
        test_loader  = test_loader,
    )
```


<https://colab.research.google.com/drive/1LDIu5IvAqQC0xujrGCff0nCaB9RfsKOJ?usp=sharing#scrollTo=c194d1e0-1942-4b40-8bb4-28ae8d68...> 25/29

```

val_accs = [results_bs[l]['best_val_acc'] for l in labels]
test_accs = [results_bs[l]['test_acc'] for l in labels]

b1 = ax.bar(x - w/2, val_accs, w, label='Best Val Acc', color='steelblue', edgecolor='black')
b2 = ax.bar(x + w/2, test_accs, w, label='Test Acc', color='darkorange', edgecolor='black')
ax.set_xticks(x); ax.set_xticklabels(labels)
ax.set_ylabel('Accuracy (%)'); ax.set_title('Val vs Test Accuracy por Batch Size', fontweight='bold')
ax.legend(); ax.grid(axis='y', alpha=0.3)
for bar, v in list(zip(b1, val_accs)) + list(zip(b2, test_accs)):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height()+0.2, f'{v:.1f}%', ha='center', va='bottom', fontsize=8)
plt.tight_layout(); plt.savefig('/content/exp_bs_bar.png', dpi=130, bbox_inches='tight'); plt.show()

```

(0): Dropout(p=0.5, inplace=False)

(1): Linear(in_features=2048, out_features=39, bias=True)

Experimento 3 — Número de Capas Descongeladas

Hipótesis: Descongelar más capas permite al modelo adaptar características de más alto nivel al dominio

de hojas vegetales, pero aumenta el riesgo de sobreajuste y el tiempo de cómputo.

Configuración	Capas entrenables	Paráms. entrenables (aprox.)
fc_only	Solo cabeza FC	~80K
layer4+fc	layer4+FC	~6M
layer3+4+fc	layer3, layer4, FC	~20M
layer2+3+4+fc	layer2, layer3, layer4, FC	~23M

Variables fijas: Adam · lr_fc=1e-3 · BS=32 · 5 épocas

```

print("=" * 60)
print(" EXPERIMENTO 3: CAPAS DESCONGELADAS")
print("=" * 60)

layer_configs = {
    'fc_only' : 'fc_only',
    'layer4+fc' : 'layer4+fc',
    'layer3+4+fc' : 'layer3+4+fc',
    'layer2+3+4+fc' : 'layer2+3+4+fc',
}
results_layers = {}

for label, ucfg in layer_configs.items():
    print(f"\n► Probando {label} ...")
    results_layers[label] = run_experiment(
        config = {'lr_fc': 1e-3, 'optimizer_name': 'Adam', 'unfreeze_config': ucfg},
        train_loader = train_loader,
        val_loader = val_loader,
        test_loader = test_loader,
    )

```

AssertionError: can only test a single process

```

# — Visualización Experimento 3 —
plot_exp_curves(results_layers, title='Capas Descongeladas')
df_layers = summary_table(results_layers, exp_name='Capas Descongeladas')

# — Tiempo de entrenamiento por configuración —
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 4))
labels = list(results_layers.keys())

times = [results_layers[l]['training_time']/60 for l in labels]
test_accs = [results_layers[l]['test_acc'] for l in labels]
x = np.arange(len(labels))

ax1.bar(x, times, color=[PALETTE[i] for i in range(len(labels))], edgecolor='black')
ax1.set_xticks(x); ax1.set_xticklabels(labels, rotation=15, ha='right')
ax1.set_ylabel('Tiempo (min)'); ax1.set_title('Tiempo de Entrenamiento', fontweight='bold')
ax1.grid(axis='y', alpha=0.3)

ax2.bar(x, test_accs, color=[PALETTE[i] for i in range(len(labels))], edgecolor='black')
ax2.set_xticks(x); ax2.set_xticklabels(labels, rotation=15, ha='right')
ax2.set_ylabel('Test Accuracy (%)'); ax2.set_title('Test Accuracy vs Capas Descongeladas', fontweight='bold')
ax2.grid(axis='y', alpha=0.3)
for bar, v in zip(ax2.patches, test_accs):
    ax2.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.2, f'{v:.1f}%', ha='center', va='bottom')

plt.tight_layout(); plt.savefig('/content/exp_layers.png', dpi=130, bbox_inches='tight'); plt.show()

```

^^^^Exception ignored in: <function _MultiProcessingDataLoaderIter.__del__ at 0x7834ee15a520>

File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in is_alive

```
Traceback (most recent call last):
```

```

Exp: File "os/10ca711b/python3.72/dist-packages/locoth/kill_data/data_loader.py", line 166, in __del__
assert self._parent_pid == os.getpid(), 'can only test a child process'    self._shutdown_workers()

```

File `/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py`, line 1669, in `_shutdown_workers`
 39 clases. Ref: <https://pytorch.org/docs/stable/nn.html#torch.nn.Lstm>
 39 clases. Ref: <https://pytorch.org/docs/stable/nn.html#torch.nn.Lstm>

39 clases. RelU y GELU deberían superar a Sigmoid/Tanh por evitar el problema de gradientes saturados.

Configuración de la arquitectura de cabecera

none ★ baseline Dropout → Linear(2048→39)

```
relu, assertionError: Dropout → Linear(2048→512) → ReLU → Dropout → Linear(512→39)
```

tanh Dropout → Linear(2048→512) → Tanh → Dropout → Linear(512→39)

```
sigmoid → Dropout → Linear(2048→512) → Sigmoid → Dropout → Linear(512→1)
```

Dropout → Linear(2048→512) → GELU → Dropout → Linear(512→39)

leaky relu → Dropout → Linear(2048→512) → LeakyReLU(0.1) → Dropout → Linear

Variables fijas: Adam · lr $1e-3$ · BS=32 · layer4+fc · 5 épocas

```
^^AssertionError^^: can only test a child process
```

```
print("=" * 60)
print(" EXPERIMENTO 4: FUNCIONES DE ACTIVACIÓN")
print("=" * 60)
```

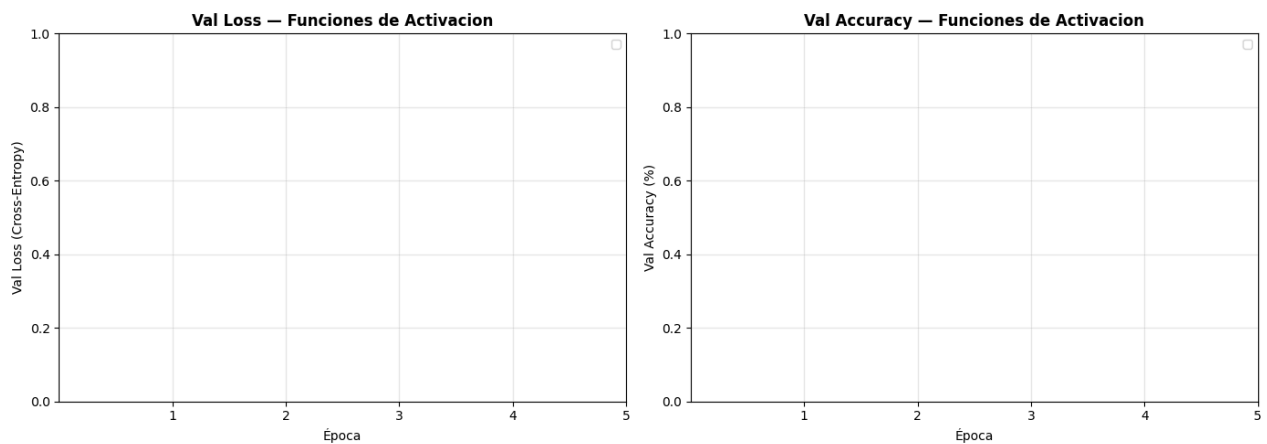
```
activation_configs = {
    'none'      : None,
    'relu'      : 'relu',
    'tanh'      : 'tanh',
    'sigmoid'   : 'sigmoid',
    'gelu'      : 'gelu',
    'leaky_relu': 'leaky_relu',
}
```

```
results_act = {}
for label, act in activation_configs.items():
    print(f"\n► Probando activación: {label} ...")
    results_act[label] = run_experiment(
        config = {
            'lr_fc' : 1e-3,
            'optimizer_name' : 'Adam',
            'unfreeze_config': 'layer4+fc',
            'head_activation': act,
            'hidden_dim' : 512,
        },
        train_loader = train_loader,
        val_loader = val_loader,
        test_loader = test_loader,
    )
```

```
# — Visualización Experimento 4 —
plot_exp_curves(results_act, title='Funciones de Activación')
df_act = summary_table(results_act, exp_name='Funciones de Activación')
```

```
# — Radar-style comparison: val_acc por época —
fig, ax = plt.subplots(figsize=(10, 5))
eps = list(range(1, EXP_EPOCHS + 1))
for i, (label, res) in enumerate(results_act.items()):
    ax.plot(eps, res['history']['val_acc'], '-o', label=label, color=PALETTE[i], lw=2, ms=6)
    # Marcar el mejor epoch
    best_e = res['best_epoch']
    ax.axvline(x=best_e, color=PALETTE[i], linestyle=':', alpha=0.4)
ax.set_xlabel('Época'); ax.set_ylabel('Val Accuracy (%)')
ax.set_title('Convergencia por Función de Activación', fontweight='bold')
ax.legend(loc='lower right', fontsize=9); ax.grid(True, alpha=0.3); ax.set_xticks(eps)
plt.tight_layout(); plt.savefig('/content/exp_act.png', dpi=130, bbox_inches='tight'); plt.show()
```

```
^ File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in is_alive
    ^assert self._parent_pid == os.getpid(), 'can only test a child process'^
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^AssertionError: can only test a child process^^
^^Exception ignored in: ^<function _MultiProcessingDataLoaderIter.__del__ at 0x7834ee15a520>
^Traceback (most recent call last):
^ File "/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py", line 1686, in __del__
    ^self._shutdown_workers()
^ File "/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py", line 1669, in _shutdown_workers
    if w.is_alive():
    ^
    ^
    ^
    AssertionError : can only test a child process^
^^^Exception ignored in: ^^<>function _MultiProcessingDataLoaderIter.__del__ at 0x7834ee15a520>^
^^Traceback (most recent call last):
^ File "/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py", line 1686, in __del__
    ^
    ^
    self.shutdown workers() File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in is alive
```



```

--(0): Dropout(p=0.5, inplace=False)
KeyError: "None of ['Configuración'] are in the columns"
/tmp/ipykernel_1791/3068817932.py in <cell line: 0>()
1 # — Visualización Experimento 4 —
2 plot_exp_curves(results_act, title='Funciones de Activacion')
----> 3 df_act = summary_table(results_act, exp_name='Funciones de Activación')
4
Ep 0/5 | Train: 3% | Val Loss=0.5710 | Val Accuracy=94.58% | 1 frames | 36/1213 [00:09<04:25, 4.43it/s]

/usr/local/lib/python3.12/dist-packages/pandas/core/frame.py in set_index(self, keys, drop, append, inplace,
verify_integrity)
6120
6121     if missing:
-> 6122         raise KeyError(f"None of {missing} are in the columns")
6123
6124     if inplace:
KeyError: "None of ['Configuración'] are in the columns"

```

✎ Experimento 5 — Optimizadores

Hipótesis: Adam y AdamW convergen más rápido gracias a los momentos adaptativos.

SGD con Nesterov puede alcanzar mejor generalización si se entrena suficientes épocas.

RMSprop es competitivo en visión computacional.

Optimizador	Descripción	Regularización
Adam  baseline	Momentos adaptativos 1er y 2do orden	weight_decay=1e-4
AdamW	Adam con decoupled weight decay	weight_decay=1e-2
SGD	Descenso de gradiente estocástico puro	weight_decay=1e-4
SGD_Nesterov	SGD + momentum=0.9 + lookahead Nesterov	weight_decay=1e-4
RMSprop	Normalización por media de gradientes cuadráticos	weight_decay=1e-4

Variables fijas: lr_fc=1e-3 · BS=32 · layer4+fc · 5 épocas

```

print("=" * 60)
print(" EXPERIMENTO 5: OPTIMIZADORES")
print("=" * 60)

optimizer_configs = {
    'Adam'      : 'Adam',
    'AdamW'     : 'AdamW',
    'SGD'       : 'SGD',
    'SGD_Nesterov': 'SGD_Nesterov',
    'RMSprop'   : 'RMSprop',
}

results_opt = {}
for label, opt_name in optimizer_configs.items():
    print(f"\n► Proband optimizador: {label} ...")
    results_opt[label] = run_experiment(
        config = {
            'lr_fc'      : 1e-3,
            'optimizer_name': opt_name,
            'unfreeze_config': 'layer4+fc',
        },
        train_loader = train_loader,
        val_loader   = val_loader,
        test_loader  = test_loader,
    )

```



```
)
```

```
# — Visualización Experimento 5 —  
plot_exp_curves(results_opt, title='Optimizadores')  
df_opt = summary_table(results_opt, exp_name='Optimizadores')
```